

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО–КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №9
дисциплины
«Объектно–ориентированное программирование»
Вариант 13

Выполнил:
Рябинин Егор Алексеевич
3 курс, группа ИВТ–б–о–23–2,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии
Воронкин Роман Александрович

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2025 г

Тема: Управление процессами в Python.

Цель: Приобретение навыков написания многозадачных приложений на языке программирования Python версии 3.13.3.

Порядок выполнения работы:

Ссылка на репозиторий:

https://github.com/bohemiaaaaa/Lab9_Object-oriented-programming

Задание №1. С использованием многопроцессности для заданного значения x найти сумму ряда S с точностью члена ряда по абсолютному значению $\varepsilon = 10^{-7}$ и произвести сравнение полученной суммы с контрольным значением функции y .

$$S = \sum_{n=1}^{\infty} \frac{1}{(2n-1)x^{2n-1}} = \frac{1}{x} + \frac{1}{3x^3} + \frac{1}{5x^5} + \dots;$$
$$x = 3; y = \frac{1}{2} \ln \frac{x+1}{x-1}.$$

Листинг программы:

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import math
from multiprocessing import Manager, Process

class SeriesProcess(Process):
    def __init__(
        self, x: float, eps: float, start_idx: int, step: int, results: dict,
        pos: int
    ) -> None:
        super().__init__()
        self.x: float = x
        self.eps: float = eps
        self.start_idx: int = start_idx
        self.step: int = step
        self.results: dict = results
        self.pos: int = pos

    def _term(self, n: int) -> float:
        try:
            return 1.0 / ((2 * n - 1) * (self.x ** (2 * n - 1)))
        except OverflowError:
            return 0.0

    def run(self) -> None:
        partial_sum: float = 0.0
        count: int = 0

        n: int = self.start_idx
        while True:
            term: float = self._term(n)
```

```

        if abs(term) < self.eps:
            break
        partial_sum += term
        count += 1
        n += self.step

    self.results[f"sum_{self.pos}"] = partial_sum
    self.results[f"count_{self.pos}"] = count

def calculate_series(x: float, eps: float, num_processes: int = 4) ->
tuple[float, int]:
    with Manager() as manager:
        results: dict = manager.dict()
        processes: list = []

        for i in range(num_processes):
            p = SeriesProcess(x, eps, i + 1, num_processes, results, i)
            processes.append(p)
            p.start()

        for p in processes:
            p.join()

        total_sum: float = sum(results[f"sum_{i}"] for i in
range(num_processes))
        total_count: int = sum(results[f"count_{i}"] for i in
range(num_processes))

        return total_sum, total_count

def get_control_value(x: float) -> float:
    return 0.5 * math.log((x + 1) / (x - 1))

```

```

● PS C:\Users\4isto\00P_lab9> python tasks/task1.py
Ряд:  $S = \sum [1/((2n-1)*x^{(2n-1)})]$ ,  $n=1..∞$ 
 $x = 3.0$ ,  $\epsilon = 1e-07$ 

Результаты:
Вычисленная сумма: 0.3465735369
Контрольное значение: 0.3465735903
Абсолютная погрешность: 5.34e-08
Учтено членов ряда: 6
Точность достигнута (погрешность <  $\epsilon$ )

```

Рисунок 1 – Результат работы программы

Тесты для написанной программы:

Листинг программы:

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import math

import pytest

from tasks.series import calculate_series, get_control_value

```

```

@pytest.mark.parametrize(
    "x, eps",
    [
        (2.0, 1e-7),
        (3.0, 1e-8),
        (5.0, 1e-6),
    ],
)
def test_calculate_series_accuracy(x: float, eps: float) -> None:
    total_sum, total_count = calculate_series(x, eps, num_processes=4)
    control_value: float = get_control_value(x)
    error: float = abs(total_sum - control_value)

    assert error < eps, f"Ошибка {error} превышает eps={eps}"
    assert total_count > 0

def test_control_value_known() -> None:
    x: float = 2.0
    expected: float = 0.5 * math.log((x + 1) / (x - 1))
    assert math.isclose(get_control_value(x), expected, rel_tol=1e-12)

def test_series_convergence_monotone() -> None:
    x: float = 3.0
    sum_coarse, _ = calculate_series(x, 1e-4, num_processes=4)
    sum_fine, _ = calculate_series(x, 1e-8, num_processes=4)
    control: float = get_control_value(x)

    assert abs(sum_fine - control) < abs(sum_coarse - control)

```

```

● PS C:\Users\4isto\OOP_lab9> pytest
===== test session starts =====
platform win32 -- Python 3.11.0, pytest-8.3.5, pluggy-1.6.0 -- C:\Program Files\Python311\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\4isto\OOP_lab9
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.9.0
collected 5 items

tests/test_task1.py::test_calculate_series_accuracy[2.0-1e-07] PASSED
tests/test_task1.py::test_calculate_series_accuracy[3.0-1e-08] PASSED
tests/test_task1.py::test_calculate_series_accuracy[5.0-1e-06] PASSED
tests/test_task1.py::test_control_value_known PASSED
tests/test_task1.py::test_series_convergence_monotone PASSED

===== 5 passed in 1.03s =====

```

Рисунок 2 – Результат работы тестов

Контрольные вопросы:

1. Как создаются и завершаются процессы в Python?

Процессы в Python создаются с помощью класса `Process` из модуля `multiprocessing`. Для создания процесса нужно создать экземпляр класса `Process`, передав в конструктор `target` – функцию, которую нужно выполнить

в отдельном процессе, и `args/kwargs` – аргументы этой функции. Запуск процесса выполняется методом `start()`. Процесс завершается автоматически после выполнения целевой функции. Для ожидания завершения процесса используется метод `join()`. Также процесс можно завершить принудительно методами `terminate()` или `kill()`.

2. В чем особенность создания классов-наследников от `Process`?

При создании классов-наследников от `Process` нужно переопределить метод `run()`. В этом методе описывается логика, которая будет выполняться в отдельном процессе. Конструктор класса-наследника должен вызывать конструктор родительского класса с помощью `super().init()`. Такой подход позволяет создавать более структурированные и объектно-ориентированные многопроцессные приложения, где каждый процесс инкапсулирует свое состояние и поведение.

3. Как выполнить принудительное завершение процесса?

Для принудительного завершения процесса в Python используются два метода: `terminate()` и `kill()`. Метод `terminate()` отправляет процессу сигнал `SIGTERM` в Unix или вызывает `TerminateProcess()` в Windows. Метод `kill()` работает аналогично, но в Unix отправляет сигнал `SIGKILL`. Эти методы позволяют завершить процесс до окончания его нормальной работы, но могут привести к утечкам ресурсов, если процесс не был подготовлен к корректному завершению.

4. Что такое процессы-демоны? Как запустить процесс-демон?

Процессы-демоны в Python – это процессы, которые автоматически завершаются при завершении родительского процесса. Они похожи на потоки-демоны и используются для фоновых задач, которые должны работать только пока работает основная программа. Запустить процесс-демон можно двумя способами: передать аргумент `daemon=True` при создании объекта `Process`, либо установить свойство `daemon` в `True` после создания процесса, но до вызова метода `start()`. Процессы-демоны не могут создавать собственные дочерние процессы.

Вывод: в ходе лабораторной работы были приобретены навыки написания многозадачных приложения на языке программирования Python версии 3.13.3.